17、开发环境再升级

全志V3S嵌入式驱动开发（开发环境再升级）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2023-08-29 15:01:15 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量3.1k Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 2

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了如何优化全志V3S开发环境，包括解决uboot启动kernel问题，开机无需输入root用户名，添加开机启动脚本，配置网卡，安装ftp和telnet服务器，以及便捷升级zImage和dtb文件的方法，提升开发效率。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

前面我们陆陆续续开发了差不多有10个驱动，涉及到网口、串口、音频和视频等几个方面。但是整个开发的效率还是比较低的。每次开发调试的时候都很麻烦，譬如说，如果是驱动代码的问题，就需要把sd卡拆下来，重新烧入；再比如说，uboot和linux之间不能连续运行，需要手动做一些配置之后，再启动linux kernel；还比如说，使用480*272的dtb文件之后，之前的网卡功能都丢失了，很不方便。所以针对这些问题，我们想办法集中处理一下，这样有利于后期的复杂驱动调试、以及上层软件开发。

目前已经升级的开发环境主要有这7个方面，

a）解决uboot不能正常启动linux kernel的问题；

b）解决开机后需要输入root用户名的问题；

c）添加开机启动脚本的问题；

d）添加网卡；

e）安装ftp服务器软件；

f）安装telnet服务器软件;

g）自由升级zImage和dtb文件。

注1： 实际使用的时候，不要删除ftp账户。假设root账户无法登录，这个时候之前创建的ftp账户还可以派上用场，可以先用ftp登录，然后su到root上面去。

注2：另外一个建议，就是大家上传可执行文件的时候，一定要先压缩再解压，这样基本肯定不会出问题。

1、解决uboot不能正常启动linux kernel的问题

要解决这个问题，关键在于增加CONFIG_BOOTCOMMAND和CONFIG_BOOTARGS这两个宏。 **因为我们使用的是v3s这颗芯片，所以在uboot上面，可以把上述两个宏添加到include/configs/sun8i.h这个文件下面，**

```
#include <configs/sunxi-common.h>

#ifdef CONFIG_BOOTCOMMAND
#undef CONFIG_BOOTCOMMAND
#endif

#define CONFIG_BOOTCOMMAND    "setenv bootm_boot_mode sec; "\
                "load mmc 0:1 0x41000000 zImage;"  \
                "load mmc 0:1 0x41800000 sun8i-v3s-licheepi-zero-with-
480x272-lcd.dtb;" \
                "bootz 0x41000000 - 0x41800000;"

#ifdef CONFIG_BOOTARGS
#undef CONFIG_BOOTARGS
#endif

#define CONFIG_BOOTARGS      "console=ttyS0,115200 panic=5 rootwait
root=/dev/mmcblk0p2 earlyprintk rw vt.global_cursor_default=0"
```

注意两个宏的设置都放在include头文件之后。当然因为这个文件发生了修改，那么整个uboot就需要重新编译一下，

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
```

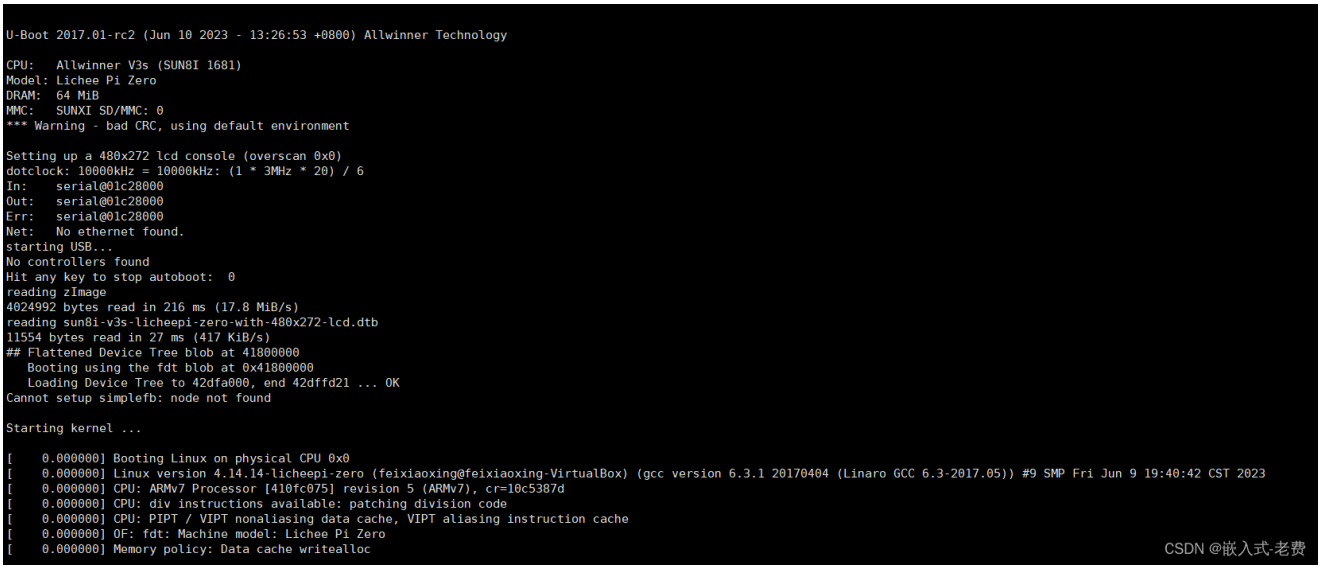
等编译结束，生成了我们需要的bin文件之后，就可以pc上插入读卡器，准备bin文件烧入了，

```
feixiaoxing@feixiaoxing-VirtualBox:~/Desktop/u-boot-3s-current$ sudo dd
if=u-boot-sunxi-with-spl.bin of=/dev/sdb bs=1024 seek=8
[sudo] password for feixiaoxing:
389+1 records in
```

```
389+1 records out
398367 bytes (398 kB, 389 KiB) copied, 1.71158 s, 233 kB/s
```

烧入的工具是利用ubuntu虚拟机里面的dd命令来完成的。为了保证烧入ok，可以继续输入一个sync命令。结束后，退出sd卡，重新把micro sd卡插入到开发板上，不出意外的话，就可以看到uboot、kernel连续启动的过程了，

Open: eead827d8458c091eec417bbd02690ac_MD5.png



如果你已经可以看到这样的内容，代表当前uboot可以持续启动linux kernel了。

2、解决开机后需要输入root用户名的问题

如果当前还处于开发阶段，每次都需要输入root用户名，其实是很令人反感的一个事情。虽然是免密登录，但是还是想着可以更方便一点。这个时候，完全可以通过文件配置的方法来解决这一问题。

```
# Put a getty on the serial port
#console::respawn:/sbin/getty -L  console 0 vt100# GENERIC_SERIAL
ttyS0::once:/bin/login root
```

在原来/etc/inittab里面注释掉getty的内容，添加login这一行，那么我们每次开机的时候就不需要手动输入root这个用户了。 并且，开机之后，自动切换到/root这个目录下面。

3、添加开机启动脚本的问题

有一些程序或者配置，我们其实是希望系统启动后可以默认执行的，比如这个时候就希望linux可以自动帮我们执行一些脚本，例如auto.sh。要实现这个目标，这个时候，可以这么做，分成三步。

第一步，在/etc/init.d/rcS最后一行添加这么一句，

```
/etc/rc.local
```

第二步，创建/etc/rc.local文件，填充下面这些内容，同时chmod +x /etc/rc.local，

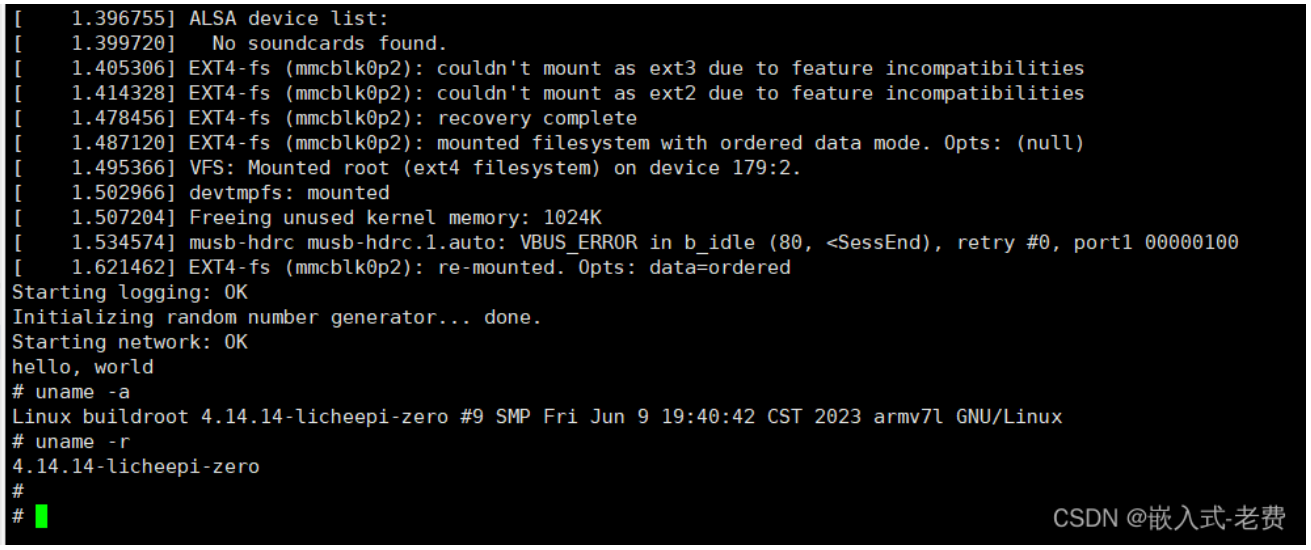
```
#!/bin/sh
/root/auto.sh &
```

第三步，创建/root/auto.sh文件，添加一点测试内容，比如

```
echo "hello, world"
```

同样auto.sh需要chmod +x auto.sh设置一下，让它变成可执行文件。为了测试我们的配置有没有效果，可以按下板子上的复位键，如果看到这些输出，大体可以认为我们的设计是ok的。

Open: 79fcd8714fe7e5b39f0670451283fdcd_MD5.png



仔细找一下，还是发现了hello, world的身影。这说明我们的配置是正确的。

4、添加网卡

观察发现，sun8i-v3s-licheepi-zero-with-480×272-lcd.dts和sun8i-v3s-licheepi-zero-with-lcd.dtsi这两个文件中，都没有关于以太网的配置。而之前sun8i-v3s-licheepi-zero-dock.dts中则包含了相关的内容，这个时候我们要做的，就是把dsun8i-v3s-licheepi-zero-dock.dts中关于网卡的配置挪到sun8i-v3s-licheepi-zero-with-lcd.dtsi当中去就可以了，

```
    aliases {
        ethernet0 = &emac;
    };
```

首先是ethernet0指向了emac，

```
&emac {
    status = "okay";
};
```

其实是emac中的status设置为okay。别看这两步很简单，但是如果不做的话，网卡就是启动不起来。 **当然有了这两点还是不够的，我们需要在/etc/network/interfaces中增加一点配置，**

```
auto lo
iface lo inet loopback
auto eth0
iface eth0 inet static
address 192.168.0.230
netmask 255.255.255.0
gateway 192.168.0.1
dns-nameservers 8.8.8.8
```

从脚本上看，系统启动后除了配置loopback之外，还需要添加eth0，并且需要进一步设置ip、netmask和gateway。这一顿操作之后，如果pc电脑还在同一网段，基本就可以和电脑之间实现ping的正常通讯了。

```
# ifconfig
eth0      Link encap:Ethernet  HWaddr 02:00:A4:52:F3:CC
          inet addr:192.168.0.230  Bcast:0.0.0.0  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:384 errors:0 dropped:0 overruns:0 frame:0
          TX packets:162 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:33241 (32.4 KiB)  TX bytes:7140 (6.9 KiB)
          Interrupt:36

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
```

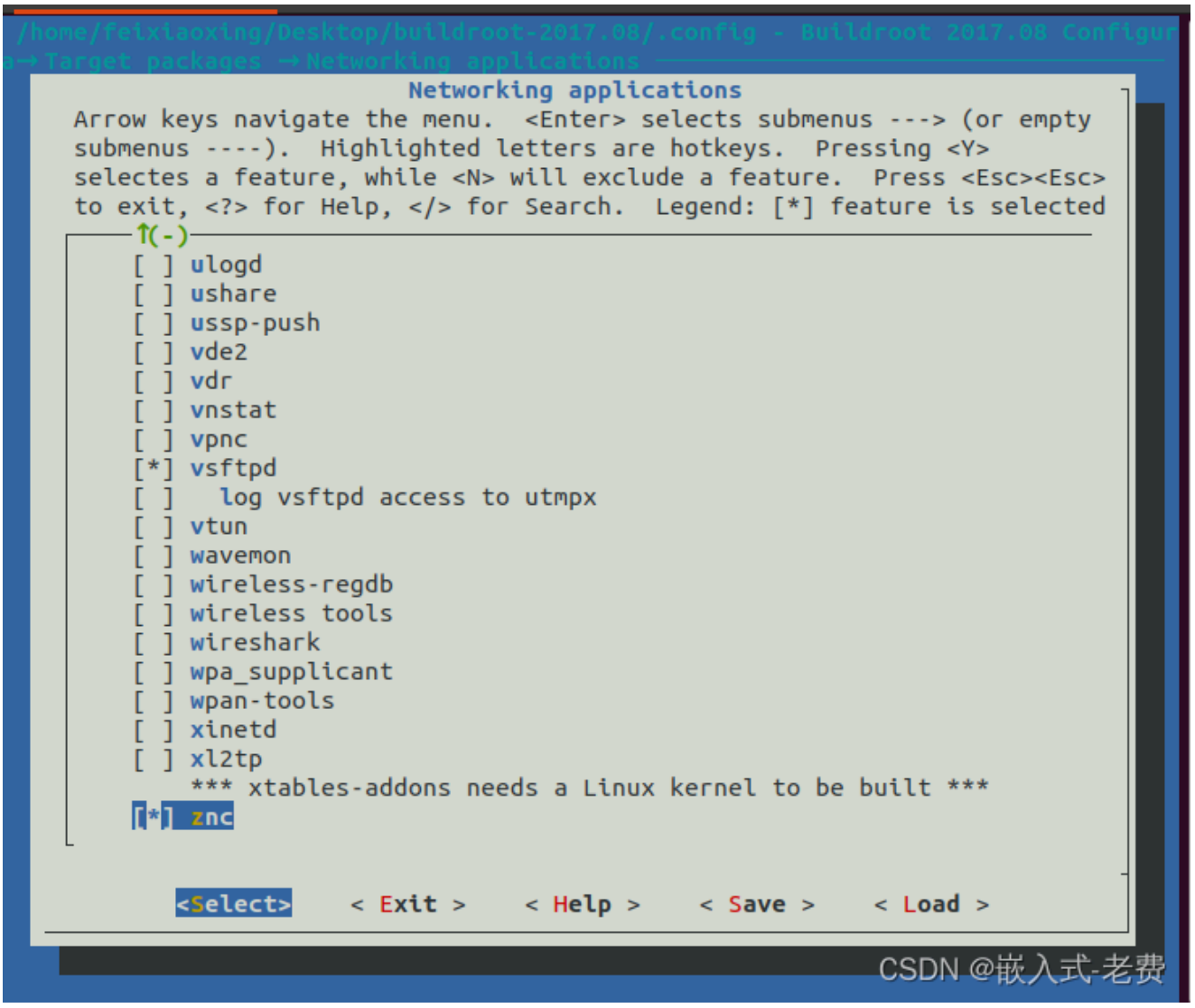
```
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

# ping 192.168.0.100
PING 192.168.0.100 (192.168.0.100): 56 data bytes
64 bytes from 192.168.0.100: seq=0 ttl=128 time=92.529 ms
64 bytes from 192.168.0.100: seq=1 ttl=128 time=12.744 ms
64 bytes from 192.168.0.100: seq=2 ttl=128 time=38.267 ms
64 bytes from 192.168.0.100: seq=3 ttl=128 time=60.146 ms
64 bytes from 192.168.0.100: seq=4 ttl=128 time=84.125 ms
64 bytes from 192.168.0.100: seq=5 ttl=128 time=80.020 ms
^C
--- 192.168.0.100 ping statistics ---
6 packets transmitted, 6 packets received, 0% packet loss
round-trip min/avg/max = 12.744/61.305/92.529 ms
```

5、安装ftp软件

嵌入式ftp，一般都会使用vsftpd这个软件。大家可以通过buildroot来进行编译，

Open: 3583559bec53f05e740d935b1262c886_MD5.png



选中了vsftpd之后，就可以开始make编译。实际处理中，如果发现buildroot下载vsftpd下载比较困难，可以帮助它下载一下，然后拷贝到./dl目录下，再编译也是可以的。

编译结束后，所有的根文件都在./output/target目录下，这里有两个文件需要拷贝。一个是usr/sbin下面的vsftpd，另外一个是etc/下面的vsftpd.conf。我们也需要把它拷贝到sd卡对应的根文件目录下。

这个vsftpd.conf需要按照下面这些内容重新配置下，如果嫌麻烦，可以直接拷贝下面这些内容，粘贴上去，

```
anonymous_enable=NO
local_enable=YES
write_enable=YES
local_umask=022
dirmessage_enable=YES
xferlog_enable=YES
connect_from_port_20=YES
ftpd_banner=Welcome to blah FTP service.
```

```
chroot_local_user=YES
chroot_list_enable=YES
chroot_list_file=/etc/vsftpd.chroot_list
listen=YES
allow_writeable_chroot=YES
```

除此之外，还需要几个操作步骤，

```
touch /etc/vsftpd.chroot_list
mkdir /usr/share/empty

useradd ftp    #增加新用户ftp，密码是ftp

mkdir /home/ftp
chown ftp:ftp /home/ftp
chmod 755 /home/ftp
```

有了这些做铺垫，直接shell输入/usr/sbin/vsftpd &之后，服务器就起来了。 为了验证是不是真的有效，可以用Filezilla客户端做一个测试，即输入192.168.0.230，用户名和密码都是ftp，端口21，连接看看是否成功，如果成功，进一步验证下文件上传、下载是否成功。

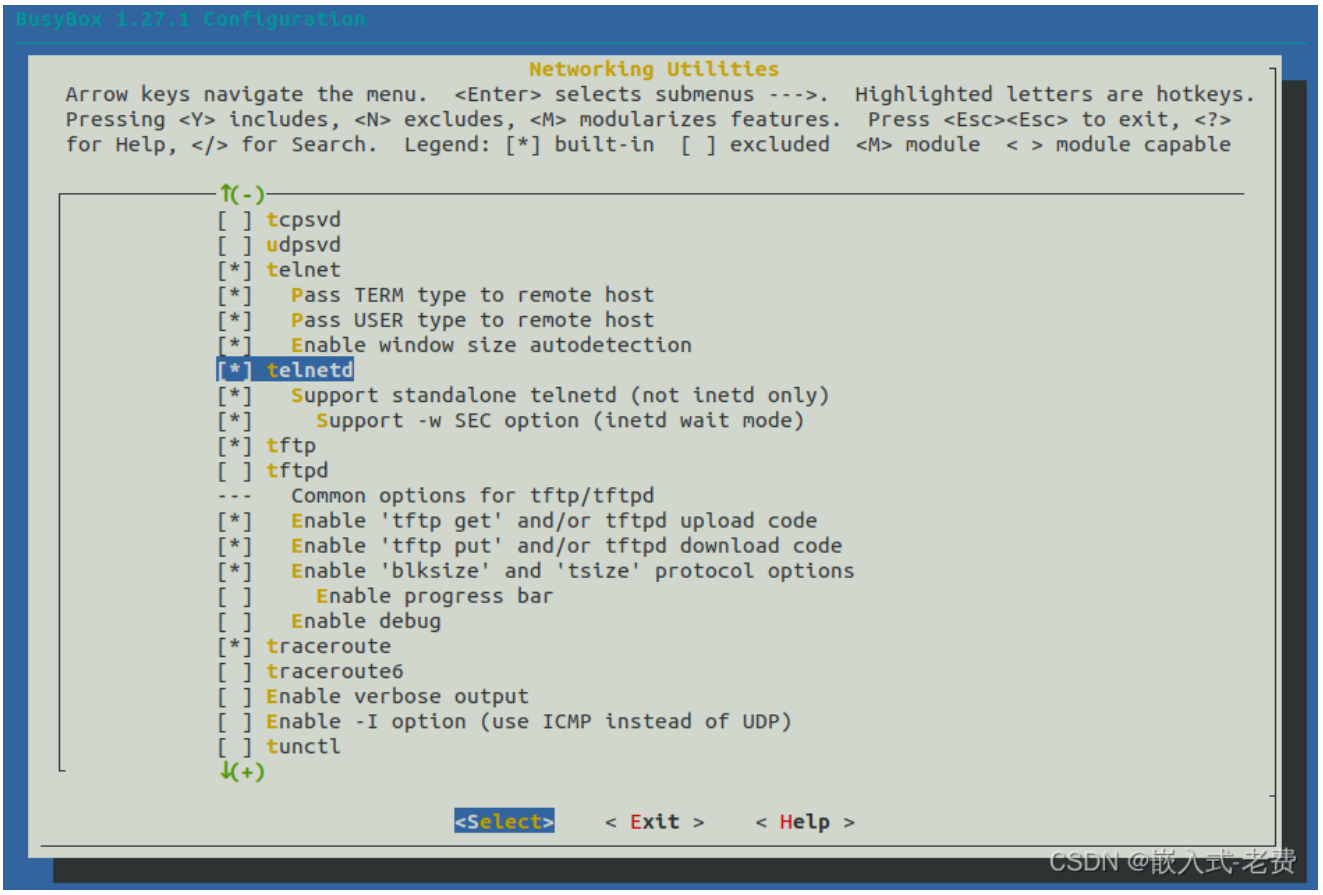
当然，现在有个小问题就是，每一次传输的时候，好像都会有一个vsftpd进程起来，这导致ps aux的时候看到好多vsftpd进程。确实有点奇怪，只是暂时还没有影响使用。实在受不了的话，只能killall vsftpd删除一下。

当然，比较优雅一点的办法，就是在vsftpd.conf里面添加一条 **max_per_ip=1** ，这也能达到我们的目的。

6、设置telnetd

之前我们和板子之间通信的时候都是走的串口ttyS0，这没问题。但是如果能够使能telnet访问到开发板，这样就显得更方便了。好在telnetd的引用也不复杂，只需要在busybox开放telnetd即可。 **首先cd到buildroot目录下，输入make busybox-menuconfig，进而在Networking Utilities中选择telnetd即可，**

Open: 98c4d43d988d5d3bf2787bbdebb48c20_MD5.png



选择好之后，直接make。这样buildroot会生成新的busybox。我们都知道，根文件的很多命令都来自于这个busybox。所以，下一步就是利用压缩的方法，把新的busybox通过ftp上传到开发板上面。 **启动方式也很直接，就是./busybox telnetd -l /bin/sh &。** 此时需要验证一下telnet的23号端口有没有打开，输入netstat -an即可看到，

```
# netstat -an
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address
State
tcp        0      0 0.0.0.0:21              0.0.0.0:*
LISTEN
tcp        0      0 0.0.0.0:23              0.0.0.0:*
LISTEN
netstat: /proc/net/tcp6: No such file or directory
netstat: /proc/net/udp6: No such file or directory
netstat: /proc/net/raw6: No such file or directory
Active UNIX domain sockets (servers and established)
Proto RefCnt Flags       Type       State         I-Node Path
unix  4      [ ]         DGRAM                      1444 /dev/log
unix  2      [ ]         DGRAM                      1949
unix  2      [ ]         DGRAM                      1448
```

有了telnet之后，测试就可以脱离串口，pc电脑也可以离开开发板，感觉人一下子解放出来，建议大家多试试telnet调试。

7、快速升级zImage和dtb文件

之前重新编译zImage和dtb文件的时候，都是需要独立烧入sd卡，使用十分不方便。现在有了网络之后，升级过程只需要两部就可以完成，

第一，将升级前的文件用ftp拷贝到开发板，注意压缩；

第二，mount /dev/mmcblk0p1 /mnt，将原来的文件删除，替换成新传上来的解压文件；

第三，重启电脑，确认功能是否ok即可。